```python
from collections import deque

goal = ((1, 2, 3),
        (4, 5, 6),
        (7, 8, 0))

# Generate all possible moves
def get_neighbors(state):
    board = [list(row) for row in
        state]

    # Find blank tile (0)
    for i in range(3):
        for j in range(3):
            if board[i][j] == 0:
                x, y = i, j

    directions = [(-1,0), (1,0), (0,-1
        ), (0,1)]
    neighbors = []

    for dx, dy in directions:
        nx = x + dx
        ny = y + dy

        if 0 <= nx < 3 and 0 <= ny < 3:
            temp = [row[:] for row in
                board]
            temp[x][y], temp[nx
                temp[nx][ny],
```

Run

```
26                temp[x][y], temp[nx][ny] =
                      temp[nx][ny],
                      temp[x][y]
27                neighbors.append(tuple
                      (tuple(row) for row in
                      temp))
28
29        return neighbors
30
31
32    # BFS Algorithm
33    def bfs(start):
34        queue = deque()
35        queue.append(start)
36
37        visited = set()
38        visited.add(start)
39
40        parent = {}
41        parent[start] = None
42
43        while queue:
44            current = queue.popleft()
45
46            if current == goal:
47                path = []
48
49                while current is no
50                    path.append(cur
51                    current =
```

Run

```
49                while current is not None:
50                    path.append(current)
51                    current =
                          parent[current]
52
53                path.reverse()
54                return path
55
56            for neighbor in get_neighbors
                  (current):
57                if neighbor not in visited:
58                    visited.add(neighbor)
59                    parent[neighbor] =
                          current
60                    queue.append(neighbor)
61
62        return None
63
64
65    # ------------ Main Program
          ------------
66
67    print("Enter Initial State (3 rows):")
68
69    start = []
70
71    for i in range(3):
72        row = tuple(map(int, input(
              ())))
```

Run

```
63
64
65  # ------------ Main Program
        ------------
66
67  print("Enter Initial State (3 rows):")
68
69  start = []
70
71  for i in range(3):
72      row = tuple(map(int, input().split
            ()))
73      start.append(row)
74
75  start = tuple(start)
76
77  solution = bfs(start)
78
79  if solution:
80      print("\nSolution Found!\n")
81
82      for step, state in enumerate
            (solution):
83          print("Step", step)
84          for row in state:
85              print(*row)
86          print()
87
88  else:
```

Run

```
Enter Initial State (3 rows):
1 2 3
4 5 6
7 0 8

Solution Found!

Step 0
1 2 3
4 5 6
7 0 8

Step 1
1 2 3
4 5 6
7 8 0


=== Code Execution Successful ===
```